

STA 5635 HW 4

Palmer Swanson

February 4, 2026

Problem 1

TISP variable selection method ith threshold operator:

$$\Theta(x, \lambda, \eta) = \begin{cases} 0 & \text{if } |x| \leq \lambda \\ \frac{x}{1+\eta} & |x| > \lambda \end{cases}$$

Penalized logistic regression:

$$L(\boldsymbol{\omega}) = \frac{1}{N} \sum_{j=1}^N \ln(1 + e^{-y_j^* \boldsymbol{\omega}^T \mathbf{x}_j}) + \lambda \sum_{i=1}^p p(\omega_i)$$

assuming $y_j^* \in \{-1, 1\}$ and algorithm iteration:

$$\boldsymbol{\omega}^{t+1} = \Theta \left(\boldsymbol{\omega}^t + \eta' X^T \left[\mathbf{y} - \frac{1}{1 + e^{-X \boldsymbol{\omega}^t}} \right], \lambda \right)$$

assuming $y_j \in \{-0, 1\}$.

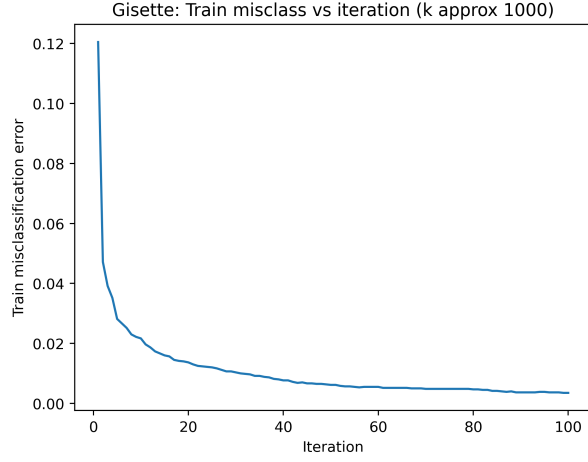
For $\eta = 0$ and 100 iterations, the first portion of question 1 is shown in Figure 1. We search for appropriate λ by performing a grid search across a range of values and selected the value that is closest to the desired number of features.

Second portion of question 1 is hown in Figure 2

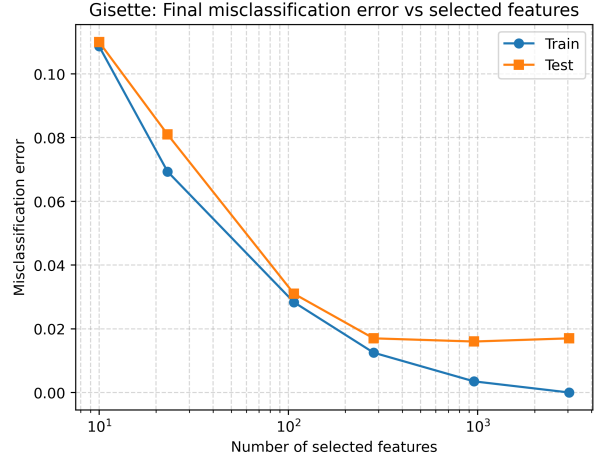
The table reporting the train and test misclassification error, the test AUC, corresponding number of features selected, values of λ , and the training time are reported in Table 1.

Dataset	Target Features	Selected Features	λ	Train Misclass.	Test Misclass.	Test AUC	Train Time (sec)
Gisette	10	10	0.196304	0.108667	0.110000	0.942920	1.844992
Gisette	30	23	0.138489	0.069333	0.081000	0.972464	1.824993
Gisette	100	107	0.086975	0.028333	0.031000	0.993964	1.795040
Gisette	300	282	0.054623	0.012500	0.017000	0.997420	1.805995
Gisette	1000	956	0.019179	0.003500	0.016000	0.998280	2.836653
Gisette	3000	3048	0.004229	0.000000	0.017000	0.997964	1.776995
Dexter	10	12	0.138489	0.116667	0.166667	0.932844	0.377510
Dexter	30	32	0.097701	0.046667	0.116667	0.959911	0.404991
Dexter	100	117	0.068926	0.003333	0.073333	0.975733	0.373484
Dexter	300	263	0.054623	0.000000	0.070000	0.979600	0.380031
Dexter	1000	1210	0.038535	0.000000	0.083333	0.968400	0.640966
Dexter	3000	1739	0.030539	0.000000	0.096667	0.966978	0.398962

Table 1: Train and Test Misclassification Errors, Test AUC, Number of Selected Features, values of λ , and training time, TISP algorithm.

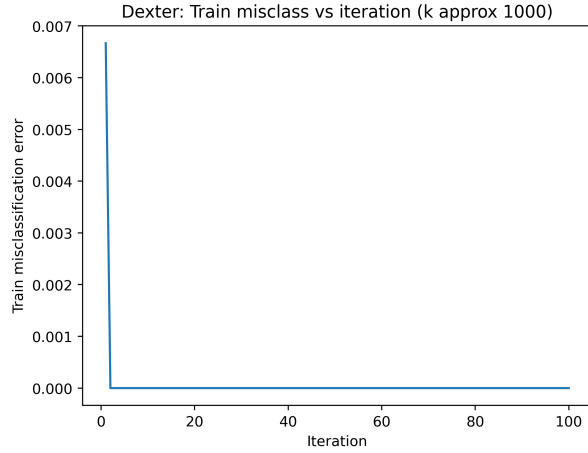


(a) Training misclassification error vs Iteration Number

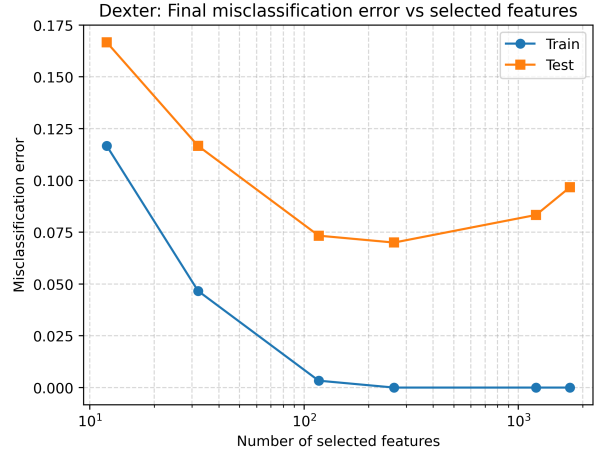


(b) Train and Test Misclassification error vs Number of Selected Features

Figure 1: Train Misclassification Error vs. TISP iteration (left) and Train and Test Misclassification Error vs. Number of Selected Features (right). Gisette Dataset.



(a) Training misclassification error vs Iteration Number



(b) Train and Test Misclassification error vs Number of Selected Features

Figure 2: Train Misclassification Error vs. TISP iteration (left) and Train and Test Misclassification Error vs. Number of Selected Features (right). Dexter Dataset.

Problem 2

FSA variable selection using logistic loss:

$$L_D(\beta) = \frac{1}{N} \sum_{i=1}^N \ln(1 + e^{-y_i \beta^T \mathbf{x}_j}) + s \|\beta\|_2^2$$

We can compute the gradient for one element with a few key facts, and a substitution:
 $u = -y_i \beta^T \mathbf{x}_j$:

$$\begin{aligned} \frac{d}{du} \log(1 + e^u) &= \frac{e^u}{1 + e^u} = \frac{1}{1 + e^{-u}} & \frac{d}{d\beta} u &= -y_i \mathbf{x}_i \\ \frac{d}{d\beta} \log(1 + e^u) &= \frac{1}{1 + e^{-u}} (-y_i \mathbf{x}_i) \end{aligned}$$

substitution gives us:

$$\frac{d}{d\beta} \log(1 + e^{-y_i \beta^T \mathbf{x}_j}) = \frac{1}{1 + e^{y_i \beta^T \mathbf{x}_j}} (-y_i \mathbf{x}_i)$$

With this, we can get the gradient for all N:

$$\nabla L_D(\beta) = \frac{-1}{N} \sum_{i=1}^N \frac{(y_i \mathbf{x}_i)}{1 + e^{y_i \beta^T \mathbf{x}_j}}$$

We also use an inverse scheduler with parameter μ :

$$M_i = k + (p - k) \max \left(0, \frac{N^{iter} - 2i}{2i\mu + N^{iter}} \right)$$

For $s = 0.001$, $\mu = 100$, and 100 iterations, the first portion of question 2 is shown in Figure 3:

At first glance, neither datasets training loss converges, which makes sense. μ is only 100 and we only have 100 iterations; μ is usually 300 and we typically have more iterations.

The table reporting train and test misclassification errors, test AUC, and training time is shown in Table 2:

With only two exceptions (the 1000 k case), TISP appears to train faster. Specifically, the ratio of the computation times FSA/TISP is greater than 1 for all but the 1000 feature case in both datasets. Did not include this table to abide by submission guidelines.

We can also include the train/test misclassification error rate vs. k for the two datasets in Figure 4:

The two methods appear to be quit similar, with only slight deviations in trajectories.

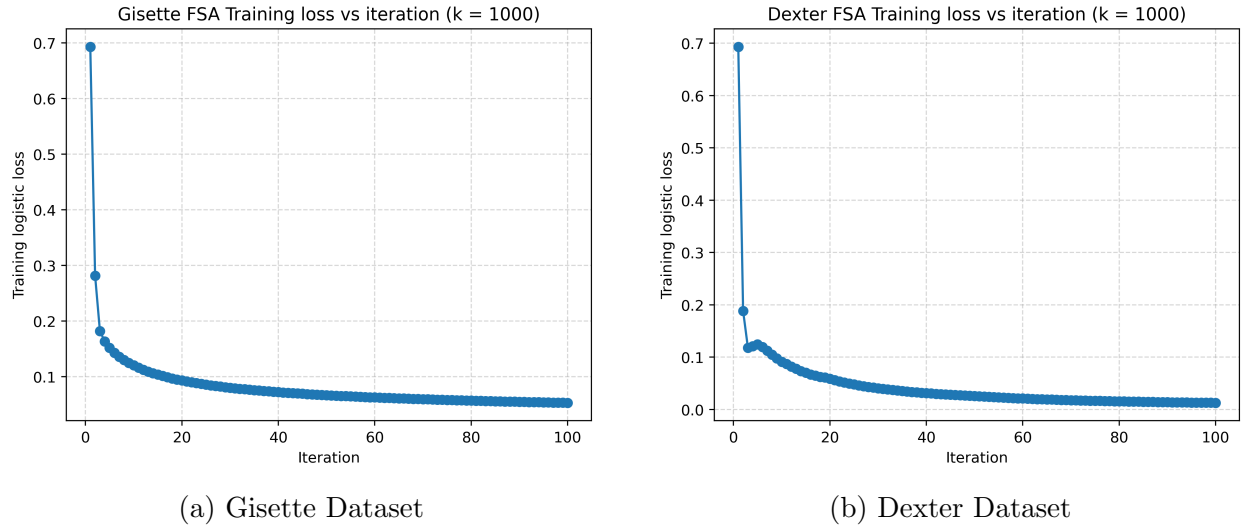
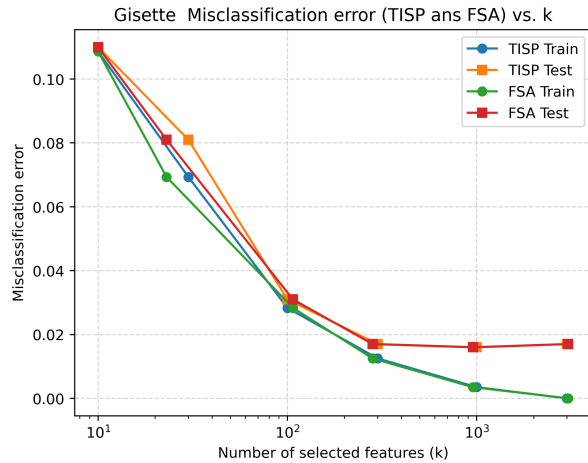


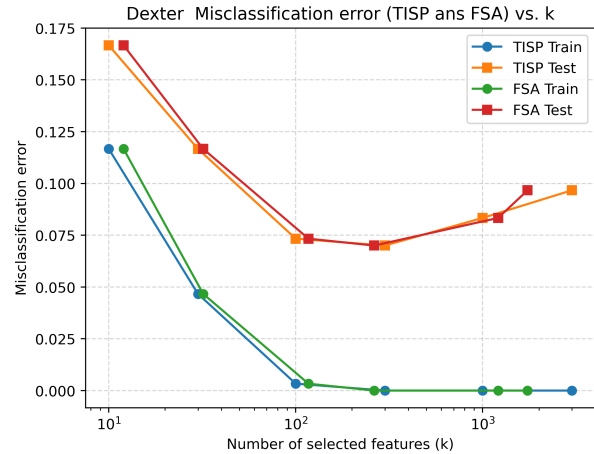
Figure 3: Plot of Training Loss vs. Iteration Number for $k = 1000$. Gisette Dataset (left), Dexter Dataset (right).

Dataset	Target k	Selected	Train Misclass.	Test Misclass.	Test AUC	Time (s)
Gisette	10	10	0.122833	0.131000	0.937842	1.916422
Gisette	30	30	0.061667	0.077000	0.981482	2.246279
Gisette	100	100	0.030333	0.030000	0.994576	2.295469
Gisette	300	300	0.017000	0.017000	0.997848	2.084871
Gisette	1000	1000	0.011667	0.017000	0.998140	2.122589
Gisette	3000	3000	0.003333	0.018000	0.997900	2.250524
Dexter	10	10	0.110000	0.153333	0.945711	0.650045
Dexter	30	30	0.040000	0.100000	0.960711	0.635123
Dexter	100	100	0.006667	0.093333	0.975067	0.578030
Dexter	300	300	0.000000	0.090000	0.971511	0.579069
Dexter	1000	1000	0.000000	0.090000	0.965289	0.553002
Dexter	3000	3000	0.000000	0.096667	0.946578	0.546987

Table 2: Train and Test Misclassification Errors, Test AUC, and training time, FSA algorithm.



(a) Gisette Dataset



(b) Dexter Dataset

Figure 4: Plot of Training Loss vs. Target Features. Gisette Dataset (left), Dexter Dataset (right).

Code

```

1 import pandas as pd
2 import numpy as np
3 from sklearn.preprocessing import StandardScaler
4 import time
5 import matplotlib.pyplot as plt
6 from sklearn.metrics import roc_auc_score, roc_curve
7
8 #Q1 datasets
9 q1_datasets = [
10     {
11         "name": "Gisette",
12         "X_train": "assignments/hw04/data/gisette_train.data",
13         "y_train": "assignments/hw04/data/gisette_train.labels",
14         "X_valid": "assignments/hw04/data/gisette_valid.data",
15         "y_valid": "assignments/hw04/data/gisette_valid.labels",
16         "missclass_1000_out":
17         "assignments/hw04/figures/gisette_misclass_1000.png",
18         "FSA_out": "assignments/hw04/figures/gisette_misclass_1000_FSA.png"
19     },
20     {
21         "name": "Dexter",
22         "X_train": "assignments/hw04/data/dexter_train.csv",
23         "y_train": "assignments/hw04/data/dexter_train.labels",
24         "X_valid": "assignments/hw04/data/dexter_valid.csv",
25         "y_valid": "assignments/hw04/data/dexter_valid.labels",

```

```

25     "missclass_1000_out":
26     "assignments/hw04/figures/dexter_misclass_1000.png",
27     "FSA_out": "assignments/hw04/figures/dexter_misclass_1000_FSA.png"
28 }
29 ]
30 #thresholding function, w is vector of length p+1
31 #I don't think we should penalize the intercept
32 #https://stats.stackexchange.com/questions/559416/ridge-regression-subtlety-on-intercept
33 #will element wise decide whether or not to select variable
34 def threshold_operator(ws, lamb):
35     #make copy of w vector
36     w_cop = ws.copy()
37
38     #https://numpy.org/devdocs/reference/generated/numpy.ones_like.html
39     #plus don't do anything to the intercept
40     #We should just get a boolean vector (F, T, T, T, T, etc)
41     mask = np.ones_like(w_cop, dtype=bool)
42     mask[0] = False
43
44     #threshold booleanwise
45     w_cop[mask & (np.abs(w_cop) <= lamb)] = 0.0
46
47     return w_cop
48
49 """
50 #Q1 parameters
51 iters = 100
52 eta = 0.0
53 features = [10, 30, 100, 300, 1000, 3000]
54
55 #we're going to be using logspace anyways
56 lambdas_to_search = np.logspace(-5, 0, 100)
57
58 results = []
59 #Q1
60 for d in q1_datasets:
61     ### get name of dataset
62     name = d["name"]
63     print(name)
64
65     ### Load datasets. use default read.csv if Dexter dataset
66     if name == "Dexter":
67         X_train = pd.read_csv(d["X_train"], header=None)
68         X_valid = pd.read_csv(d["X_valid"], header=None)
69     else:
70         X_train = pd.read_csv(d["X_train"], delim_whitespace=True, header=None)

```

```

71 X_valid=pd.read_csv(d["X_valid"],delim_whitespace=True,header=None)
72
73 y_train=pd.read_csv(d["y_train"],header=None).values.ravel()
74 y_valid=pd.read_csv(d["y_valid"],header=None).values.ravel()
75
76 #convert labels to 0/1 for later
77 y_train_converted_01=(y_train==1).astype(int)
78 y_valid_converted_01=(y_valid==1).astype(int)
79
80 #standardize
81 scaler=StandardScaler()
82 X_train_scaled=scaler.fit_transform(X_train)
83 X_test_scaled=scaler.transform(X_valid)
84
85 #add column of 1s to augment w
86 X_train_tilde=np.column_stack([np.ones(len(X_train_scaled)),
      X_train_scaled])
87 X_valid_tilde=np.column_stack([np.ones(len(X_test_scaled)),X_test_scaled])
88
89 #get N, p
90 N,p=X_train_tilde.shape
91
92 #from notes, eta' can be 1/N
93 eta_prime=1.0/N
94
95 #ok so what are we doing here. We can to find a lambda that gets us the
      closest to some number
96 #of features. first thing we need to do is loop over the number of possible
      features.
97 #going to be at least 2 loops
98 #basically a grid search over the possible lambda values
99 for k in features:
100     print(k)
101     best_lambda=None
102     best_delta=None
103     best_selected_vars=None
104
105     #this loops just searches for the best lambda
106     for j in lambdas_to_search:
107         print(j)
108
109         #initialize w to 0s
110         w=np.zeros(p)
111
112         #loop over the number of iterations
113         for i in range(iters):
114             #get predictor

```

```

115 z=X_train_tilde@w
116
117 #build fractional part
118 frac=1.0/(1.0+np.exp(-z))
119
120 #build y-fractional part
121 difference=y_train_converted_01-frac
122
123 #get the update portion
124 update_port=X_train_tilde.T@difference
125
126 #temporary new omega and do the thresholding
127 temp=w+(eta_prime*update_port)
128 w=threshold_operator(temp,j)
129
130 #need to count how many variables were selected
131 #don't count intercept and give a small tolerance
132 #check to see if we are close to the desired number of features
133 k_selected=int(np.sum(np.abs(w[1:]))>1e-10)
134 delta=abs(k_selected-k)
135
136 #if we don't have a best distance, or if the gap is smaller than the
    previous
137 #best, replace gap, lambda, number of variables selected
138 if (best_delta is None) or (delta<best_delta):
139     best_delta=delta
140     best_lambda=j
141     best_selected_vars=k_selected
142
143 #ok so now we have the lambdas. Use them to run this again
144 #now loop over everything again to time plot and get misclass rates
145 w=np.zeros(p)
146
147 #need to store misclass error only when trying to select 1000 features
148 misclass_1000=[]
149
150 start=time.time()
151 for t in range(iters):
152     z=X_train_tilde@w
153     frac=1.0/(1.0+np.exp(-z))
154     difference=(y_train_converted_01-frac)
155     update_port=X_train_tilde.T@difference
156     temp=w+(eta_prime*update_port)
157     w=threshold_operator(temp,best_lambda)
158
159 #need to track within this iterations loop
160 #now how to do that

```



```

161     if k==1000:
162         z_new=X_train_tilde@w
163         p_train_1=np.exp(-np.logaddexp(0.0,-z_new))
164         p_train_0=np.exp(-np.logaddexp(0.0,z_new))
165         odds_train=p_train_1/p_train_0
166         yhat_train=(odds_train>1.0).astype(int)
167         train_err=np.mean(yhat_train!=y_train_converted_01)
168
169         #append to misclass_1000
170         misclass_1000.append(train_err)
171
172         train_time=time.time()-start
173
174         k_selected=int(np.sum(np.abs(w[1:])>1e-10))
175
176         #train and test misclassification errors
177         z_train=X_train_tilde@w
178         z_test=X_valid_tilde@w
179
180         #get probabilities for 1 and 0
181         p_train_1=np.exp(-np.logaddexp(0.0,-z_train))
182         p_train_0=np.exp(-np.logaddexp(0.0,z_train))
183
184         p_test_1=np.exp(-np.logaddexp(0.0,-z_test))
185         p_test_0=np.exp(-np.logaddexp(0.0,z_test))
186
187         #compute the odds of train and test
188         odds_train=p_train_1/p_train_0
189         odds_test=p_test_1/p_test_0
190
191         #make predictions. Predict 1 if Odds > 1
192         #as int makes things nicer
193         yhat_train=(odds_train>1.0).astype(int)
194         yhat_test=(odds_test>1.0).astype(int)
195
196         #misclassification error
197         train_err=np.mean(yhat_train!=y_train_converted_01)
198         test_err=np.mean(yhat_test!=y_valid_converted_01)
199
200         #compute AUC
201         test_auc=roc_auc_score(y_valid_converted_01,X_valid_tilde@w)
202
203         #plotting the misclassification error vs iterations for 1000 features
204         if k==1000:
205             plt.figure()
206             plt.plot(np.arange(1, iters+1), misclass_1000)
207             plt.xlabel("Iteration")

```

```

208 plt.ylabel("Train misclassification error")
209 plt.title(f"{name}: Train misclass vs iteration (k approx 1000)")
210 plt.savefig(d["missclass_1000_out"], dpi=400, bbox_inches="tight")
211 plt.close()
212 #plt.show()
213
214 #append to results
215 results.append({
216     "dataset": name,
217     "target_features": k,
218     "selected_features": k_selected,
219     "selected_lambda": best_lambda,
220     "train_misclass": train_err,
221     "test_misclass": test_err,
222     "test_auc": test_auc,
223     "train_time_sec": train_time
224 })
225
226 #df for both Gisette and Dexter
227 df_q1 = pd.DataFrame(results)
228
229 #rename columns. LATEX wasn't liking the column names
230 df_q1_latex = df_q1.rename(columns={
231     "dataset": "Dataset",
232     "target_features": "Target Features",
233     "selected_features": "Selected Features",
234     "selected_lambda": r"$\lambda$",
235     "train_misclass": "Train Misclass.",
236     "test_misclass": "Test Misclass.",
237     "test_auc": "Test AUC",
238     "train_time_sec": "Train Time (sec)"
239 })
240
241 #need to send this table to LATEX
242 LATEX_table = df_q1_latex.to_latex(
243     index=False,
244     caption=None,
245     label=None,
246     column_format="c"*df_q1_latex.shape[1],
247     escape=False
248 )
249
250 #save latex table
251 with open("assignments/hw04/output/misclass_table.tex", "w") as f:
252     f.write(LATEX_table)
253
254 #plot the final train and test misclassification error vs # of selected features

```

```

255 #using a semilog axis
256 for name in df_q1["dataset"].unique():
257     #create filename to save plot
258     outpath = f"assignments/hw04/figures/{name}_misclass_vs_features.png"
259
260     #select the results for a given dataset
261     sub = df_q1[df_q1["dataset"] == name].copy()
262
263     #works better sorted
264     #sub = sub.sort_values("selected_features")
265
266     plt.figure()
267     plt.semilogx(sub["selected_features"].to_numpy(),
268                 sub["train_misclass"].to_numpy(), marker="o", label="Train")
269     plt.semilogx(sub["selected_features"].to_numpy(),
270                 sub["test_misclass"].to_numpy(), marker="s", label="Test")
271
272     plt.xlabel("Number of selected features")
273     plt.ylabel("Misclassification error")
274     plt.title(f"{name}: Final misclassification error vs selected features")
275     plt.legend()
276     plt.grid(True, which="both", linestyle="--", alpha=0.5)
277     plt.savefig(outpath, dpi=400, bbox_inches="tight")
278     plt.close()
279     #plt.show()
280
281     """
282     #Q2, using same datasets
283     q2_datasets = q1_datasets
284
285     s = 0.001
286     mu = 100
287     iters = 100
288     features = [10, 30, 100, 300, 1000, 3000]
289     #eta=20 was too big. Overflow error
290     eta = 0.1
291
292     results = []
293
294     for d in q2_datasets:
295         #get name of dataset
296         name = d["name"]
297         print(name)
298
299         #Load datasets. use default read.csv if Dexter dataset
300         if name == "Dexter":
301             X_train = pd.read_csv(d["X_train"], header=None)

```

```

300     X_valid = pd.read_csv(d["X_valid"], header=None)
301 else:
302     X_train = pd.read_csv(d["X_train"], delim_whitespace=True, header=None)
303     X_valid = pd.read_csv(d["X_valid"], delim_whitespace=True, header=None)
304
305     #need to check that y is +1 or -1. If loop?
306     y_train = pd.read_csv(d["y_train"], header=None).values.ravel()
307     y_valid = pd.read_csv(d["y_valid"], header=None).values.ravel()
308
309     #need to force these ya values to be -1/+1
310     #https://numpy.org/devdocs/reference/generated/numpy.where.html
311     y_train = np.where(y_train > 0, 1.0, -1.0)
312     y_valid = np.where(y_valid > 0, 1.0, -1.0)
313
314     #convert labels to 0/1 for likelihood to work with AUC
315     y_train_converted_01 = (y_train == 1).astype(int)
316     y_valid_converted_01 = (y_valid == 1).astype(int)
317
318     #standardize, not forgetting this part this time
319     scaler = StandardScaler()
320     X_train_scaled = scaler.fit_transform(X_train)
321     X_test_scaled = scaler.transform(X_valid)
322
323     #add column of 1s to augment w
324     X_train_tilde = np.column_stack([np.ones(len(X_train_scaled)),
325     X_train_scaled])
326
327     X_valid_tilde = np.column_stack([np.ones(len(X_test_scaled)), X_test_scaled])
328
329     #get N, p
330     N, p = X_train_tilde.shape
331
332     training_loss_1000 = []
333
334     #loop over the number of possible features
335     for feat in features:
336         #initialize to 0
337         beta = np.zeros(p)
338
339         start = time.time()
340
341         for i in range(iters):
342             #get predictor
343             z = X_train_tilde @ beta
344             yz = y_train * z
345
346             #need numpy function I found
347             #https://numpy.org/devdocs/reference/generated/numpy.log1p.html

```

```

346     #adding penalty component
347     exponent_part = np.exp(-yz)
348     log_part = np.log1p(exponent_part)
349     penalty = s*np.sum(beta[1:]**2)
350     loss = np.mean(log_part) + penalty
351
352     #keep track of the 1000 feature case
353     if feat == 1000:
354         training_loss_1000.append(loss)
355
356     #no sparsity penalty
357     #compute gradient. Per quiz and notes, FSA does NOT use a sparsity
penalty for feature
358     #selection
359     part1 = 1.0 / (1.0 + np.exp(yz))
360     grad = -(1/N)*(X_train_tilde.T @ (y_train*part1))
361
362     #penalty component, not penalizing intercept
363     pen_grad = 2*s*beta
364     pen_grad[0] = 0.0
365
366     grad = grad + pen_grad
367
368     #update Beta using gradient step
369     beta = beta - (eta*grad)
370
371     #inverse scheduler
372     denom = (2.0*i*mu) + iters
373     numer = (iters - (2*i))
374     ratio = numer/denom
375
376     #get maximum of 0 or the ratio. Exclude the intercept
377     p_feat = p-1
378     max_portion = max(0, ratio)
379     Mi = np.round(feat + (p_feat-feat)*max_portion)
380     Mi = int(Mi)
381
382     #need to keep the top Mi variables, not including intercept
383     #sort. Keep the largest Mi.
384     #beta length p. beta[0] = intercept. beta[1:] feature coefficients
385     #Mi = # of features permitted at this iteration
386     #https://numpy.org/doc/stable/reference/generated/numpy.argsort.html
387     b = beta[1:]
388     abs_b = abs(b)
389
390     #first entries in sort will be the smallest. Need to set all but
the last

```

```

391         #Mi to 0 aka we need to set the first p_feat-Mi to 0
392         sort = np.argsort(abs_b)
393         b[sort[:-Mi]] = 0.0
394         beta[1:] = b
395         #should end up with something that looks like
396         #beta = [1.0, 0.0, 3.0, 5.0, -3.0, 0.0, 0.0, 0.0,...]
397
398     end = time.time()
399     run_time = end-start
400
401     #gotta be a better way to do this stuff
402     #need to make predictions now for misclass and AUC
403     z_train = X_train_tilde @ beta
404     z_test = X_valid_tilde @ beta
405
406     #get probabilities for 1 and 0
407     p_train_1 = np.exp(-np.logaddexp(0.0, -z_train))
408     p_train_0 = np.exp(-np.logaddexp(0.0, z_train))
409
410     p_test_1 = np.exp(-np.logaddexp(0.0, -z_test))
411     p_test_0 = np.exp(-np.logaddexp(0.0, z_test))
412
413     #compute the odds of train and test
414     odds_train = p_train_1/p_train_0
415     odds_test = p_test_1/p_test_0
416
417     #make predictions. Predcit 1 if Odds > 1
418     #as.int makes things nicer
419     yhat_train = (odds_train > 1.0).astype(int)
420     yhat_test = (odds_test > 1.0).astype(int)
421
422     #misclassification error
423     train_err = np.mean(yhat_train != y_train_converted_01)
424     test_err = np.mean(yhat_test != y_valid_converted_01)
425
426     #compute AUC
427     test_auc = roc_auc_score(y_valid_converted_01, X_valid_tilde@beta)
428
429     #compute how many variables are non 0 aka selected
430     selected = int(np.sum(beta[1:] != 0))
431     print(selected)
432
433     #append to results
434     results.append({
435         "dataset": name,
436         "target_features": feat,
437         "selected_features": selected,

```

```

438         "train_misclass": train_err,
439         "test_misclass": test_err,
440         "test_auc": test_auc,
441         "train_time_sec": run_time
442     })
443
444     #plot the training loss vs iteration number
445     plt.figure()
446     plt.plot(range(1, len(training_loss_1000) + 1), training_loss_1000,
447              marker="o")
448     plt.xlabel("Iteration")
449     plt.ylabel("Training_logistic_loss")
450     plt.title(f"{name}_FSA_Training_loss_vs_iteration_(k=1000)")
451     plt.grid(True, linestyle="--", alpha=0.5)
452     plt.savefig(d["FSA_out"], dpi=400, bbox_inches="tight")
453     plt.close()
454     #plt.show()
455
456     #df for both Gisette and Dexter
457     df_q2 = pd.DataFrame(results)
458
459     #renaming this table for LATEX
460     df_q2_latex = df_q2.rename(columns={
461         "dataset": "Dataset",
462         "target_features": "Target_k",
463         "selected_features": "Selected",
464         "train_misclass": "Train_Misclass.",
465         "test_misclass": "Test_Misclass.",
466         "test_auc": "Test_AUC",
467         "train_time_sec": "Time(s)"
468     })
469
470     #need to send this table to LATEX
471     LATEX_table = df_q2_latex.to_latex(
472         index=False,
473         caption=None,
474         label=None,
475         column_format="c" * df_q2_latex.shape[1],
476         escape=False
477     )
478
479     #save latex table
480     with open("assignments/hw04/output/misclass_table_q2.tex", "w") as f:
481         f.write(LATEX_table)
482
483     #need to plot the train and test errors vs k
484     for name in df_q2["dataset"].unique():

```

```

484     outpath = f"assignments/hw04/figures/{name}_misclass_TISP_vs_FSA_.png"
485     #make copy of the df
486     sub1 = df_q1[df_q1["dataset"] == name].copy()
487     sub2 = df_q2[df_q2["dataset"] == name].copy()
488
489     plt.figure()
490     #TISP
491     plt.semilogx(sub2["selected_features"].to_numpy(),
492     sub1["train_misclass"].to_numpy(), marker="o", label="TISP_Train")
493     plt.semilogx(sub2["selected_features"].to_numpy(),
494     sub1["test_misclass"].to_numpy(), marker="s", label="TISP_Test")
495     #FSA
496     plt.semilogx(sub1["selected_features"].to_numpy(),
497     sub1["train_misclass"].to_numpy(), marker="o", label="FSA_Train")
498     plt.semilogx(sub1["selected_features"].to_numpy(),
499     sub1["test_misclass"].to_numpy(), marker="s", label="FSA_Test")
500
501     plt.xlabel("Number_of_selected_features_(k)")
502     plt.ylabel("Misclassification_error")
503     plt.title(f"{name}_Misclassification_error_(TISP_and_FSA)_vs._k")
504     plt.legend()
505     plt.grid(True, linestyle="--", alpha=0.5)
506     plt.savefig(outpath, dpi=400, bbox_inches="tight")
507     plt.close()
508
509     #want to compare the computation time for the two methods.
510     #not included in the writeup, but I did it.
511     df_q1["method"] = "TISP"
512     df_q2["method"] = "FSA"
513     df_time = pd.concat([df_q1, df_q2], ignore_index=True)
514
515     #need to pivot. Chaining this stuff together like the pipe operator in R
516     df_time = (
517         df_time[["dataset", "target_features", "method", "train_time_sec"]]
518         .pivot(
519             index=["dataset", "target_features"],
520             columns="method",
521             values="train_time_sec"
522         )
523         .assign(
524             FSA_over_TISP=lambda x: (x["FSA"] / x["TISP"]).round(2)
525         )
526         .reset_index()
527     )
528
529     #rename columns
530     df_time_latex = df_time.rename(columns={

```



```

527     "dataset": "Dataset",
528     "target_features": "Target_$$$k$",
529     "TISP": "TISP_Time(s)",
530     "FSA": "FSA_Time(s)",
531     "FSA_over_TISP": "FSA_/TISP"
532 })
533
534 latex_table = df_time_latex.to_latex(
535     index=False,
536     escape=False,
537     float_format="%.2f",
538     column_format="c" * df_time_latex.shape[1]
539 )
540
541 with open("assignments/hw04/output/time_comparison.tex", "w") as f:
542     f.write(latex_table)

```